Gödel Machines: *past - present - future*

Nil Geisweiller



BGI Labs

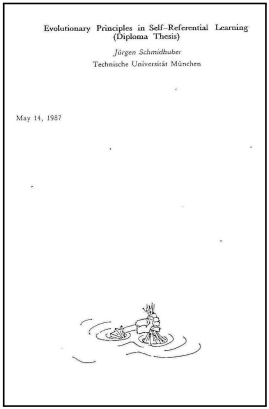


Evolutionary Principles in Self-Referential Learning (1987)



Jürgen Schmidhuber

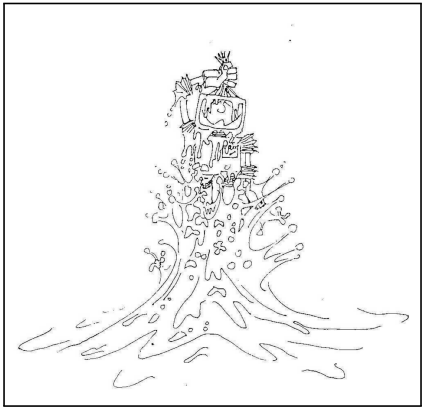
(Photo: ITU/R. Farrell, CC BY 2.0)



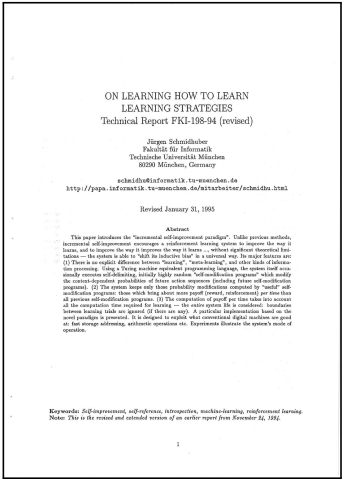
Evolutionary Principles in Self–Referential Learning
(Diploma Thesis)

Jürgen Schmidhuber
Technische Universität München

May 14, 1987



On Learning How to Learn Learning Strategies (1994)



ON LEARNING HOW TO LEARN
LEARNING STRATEGIES
Technical Report FKI-198-94 (revised)

Jürgen Schmidhuber
Fakultät für Informatik
Technische Universität München
80290 München, Germany

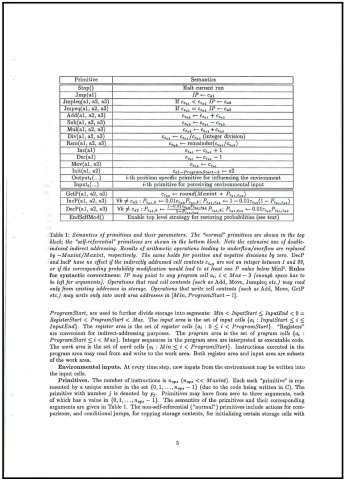
schmidhu@informatik.tu-muenchen.de
http://papa.informatik.tu-muenchen.de/mitarbeiter/schmidhu.html

Revised January 31, 1995

Abstract

This paper introduces the "incremental self-improvement paradigm". Unlike previous methods, incremental self-improvement encourages a reinforcement learning system to improve the way it learns, and to improve the way it improves the way it learns ... without significant theoretical limitations — the system is able to "shift its inductive bias" in a universal way. Its major features are: (1) There is no explicit difference between "learning", "meta-learning", and other kinds of information processing. Using a Turing machine equivalent programming language, the system itself occasionally executes self-delimiting, initially highly random "self-modification programs" which modify its own probability distribution (and thus the way it learns) and/or its own problem solving program(s). (2) The system keeps only those probability modifications computed by "useful" self-modification programs: those which bring about more payoff (reward, reinforcement) per time than all previous self-modification programs. (3) The computation of payoff per time takes into account all the computation time required for learning — the entire system life is considered; boundaries between learning trials are ignored (if there are any). A particular implementation based on the novel paradigm is presented. It is designed to exploit what conventional digital machines are good at: fast storage addressing, arithmetic operations etc. Experiments illustrate the system's mode of operation.

Keywords: *Self-improvement, self-reference, introspection, machine-learning, reinforcement learning.*
Note: *This is the revised and extended version of an earlier report from November 14, 1994.*



Primitive	Semantics
Stop()	Halt current run
Jump(a1)	$IP \leftarrow c_{a1}$
Jzero(a1, a2, a3)	If $c_{a2} < c_{Mid}$: $IP \leftarrow c_{a3}$
Jnzero(a1, a2, a3)	If $c_{a1} \geq c_{Mid}$: $IP \leftarrow c_{a3}$
Add(a1, a2, a3)	$c_{a3} \leftarrow c_{a1} + c_{a2}$
Sub(a1, a2, a3)	$c_{a3} \leftarrow c_{a1} - c_{a2}$
Mul(a1, a2, a3)	$c_{a3} \leftarrow c_{a1} * c_{a2}$
Div(a1, a2, a3)	$c_{a3} \leftarrow c_{a1}/c_{a2}$ (integer division)
Rem(a1, a2, a3)	$c_{a3} \leftarrow$ remainder(c_{a1}/c_{a2})
Inc(a1)	$c_{a1} \leftarrow c_{a1} + 1$
Dec(a1)	$c_{a1} \leftarrow c_{a1} - 1$
Mov(a1, a2)	$c_{a2} \leftarrow c_{a1}$
Init(a1, a2)	c_{a1} ← ProgramStart + a2
Output(a1, ...)	ith problem specific primitive for influencing the environment
Input(a1, ...)	ith primitive for perceiving environmental input
Get(a1, a2, a3)	$c_{a3} \leftarrow$ round(Mantissa * $2^{Exponent}$)
IncP(a1, a2, a3)	$\forall k \neq c_{a2} : P_{c_{a1},k} \leftarrow 0.01 c_{a3} P_{c_{a1},k}$; $P_{c_{a1},c_{a2}} \leftarrow 1 - 0.01 c_{a3} (1 - P_{c_{a1},c_{a2}})$
DecP(a1, a2, a3)	$\forall k \neq c_{a2} : P_{c_{a1},k} \leftarrow \frac{1-0.01 c_{a3} P_{c_{a1},c_{a2}}}{1-P_{c_{a1},c_{a2}}} P_{c_{a1},k}$; $P_{c_{a1},c_{a2}} \leftarrow 0.01 c_{a3} P_{c_{a1},c_{a2}}$
EnfSelfMod()	Enable top level strategy for executing probabilities (see text)

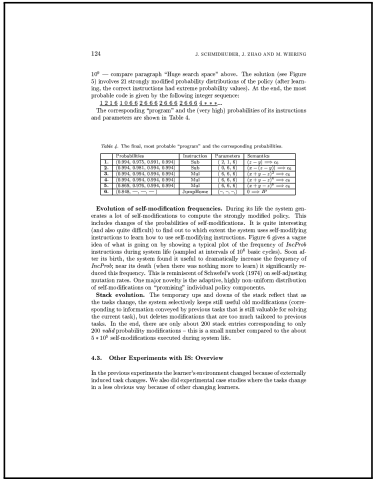
Table 1: *Semantics of primitives and their parameters. The "normal" primitives are shown in the top block; the "self-referential" primitives are shown in the bottom block. Note the extensive use of double-indexed indirect addressing. Results of arithmetic operations leading to underflow/overflow are replaced by −Maxint/Maxint, respectively. The same holds for positive and negative divisions by zero. IncP and DecP have no effect if the indirectly addressed cell contents $c_{c_{a1}}$ are not an integer between 1 and 99, or if the corresponding probability modification would lead to at least one P value below MinP. Broken* for syntactic correctness: *IP may point to any program cell ($c_i < Mem - 3$ (except space has to be left for arguments). Operations that read cell contents (such as Add, Move, Jumpnz etc.) may read only from existing addresses in storage. Operations that write cell contents (such as Add, Move, Get etc.) may write only into work area addresses in [Min, ProgramStart − 1).*

ProgramStart, are used to further divide storage into segments: $Min < InputStart \leq InputEnd < 0 = RegisterStart < ProgramStart < Mem$. The *input area* is the set of input cells $\{a_i : InputStart \leq i \leq InputEnd\}$. The *register area* is the set of *register cells* ($a_i : 0 \leq i < ProgramStart$). "Registers" are convenient for indirect-addressing purposes. The *program area* is the set of *program cells* ($a_i : ProgramStart \leq i < Mem$). Integer sequences in the program area are interpreted as executable code. The *work area* is the set of *work cells* ($a_i : Min \leq i < ProgramStart$). Instructions executed in the program area may read from and write to the work area. Both register area and input area are subsets of the work area.

Environmental inputs. At every time step, new inputs from the environment may be written into the input cells.

Primitives. The number of instructions is n_{pr} ($n_{pr} << Maxint$). Each such "primitive" is represented by a unique number in the set $\{0, 1, \ldots, n_{pr} - 1\}$ (due to the code being written in C). The primitive with number j is denoted by p_j. Primitives may have from zero to three arguments, each of which has a value in $\{0, 1, \ldots, n_{pr} - 1\}$. The semantics of the primitives and their corresponding arguments are given in Table 1. The non-self-referential ("normal") primitives include actions for comparisons, and conditional jumps, for copying storage contents, for initializing certain storage cells with

Shifting Inductive Bias with Success-Story Algorithm (1997)



124 J. SCHMIDHUBER, J. ZHAO AND M. WIERING

10^9 — compare paragraph "Huge search space" above. The solution (see Figure 5) involves 21 strongly modified probability distributions of the policy (after learning, the correct instructions had extreme probability values). At the end, the most probable code is given by the following integer sequence:

1 2 1 0, 1 0 0, 2 0 0, 2 0 0, 2 0 0, 0 x = x.

The corresponding "program" and the (very high) probabilities of its instructions and parameters are shown in Table 4.

Table 4. The final, most probable "program" and the corresponding probabilities.

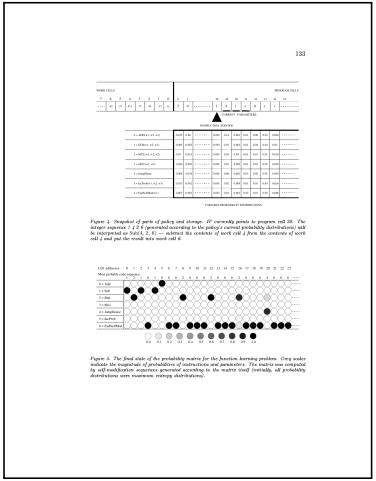
	Probabilities	Instruction	Parameters	Semantics
1	[0.994, 0.994, 0.994, 0.994]	Shift	(-1, 1, 0)	$(x \leftarrow y)$; $\Longrightarrow$ ca.
2	[0.994, 0.994, 0.994, 0.994]	Mul	(-1, 0, 1)	$(x \leftarrow x * y)$; $\Longrightarrow$ ca.
3	[0.994, 0.994, 0.994, 0.994]	Mul	(-1, 0, 1)	$(x \leftarrow x * y)$; $\Longrightarrow$ ca.
4	[0.994, 0.994, 0.994, 0.994]	Mul	(-1, 0, 1)	$(x \leftarrow x * y)$; $\Longrightarrow$ ca.
5	[0.994, 0.994, 0.994, 0.994]	Mul	(-1, 0, 1)	$(x \leftarrow x * y)$; $\Longrightarrow$ ca.
6	[0.048, —, —, —]	JumpHome	(—, —, —)	if $nil \Rightarrow 0^*$

Evolution of self-modification frequencies. During its life the system generates a lot of self-modifications to comprise the strongly modified policy. This includes changes of the probabilities of self-modifications. It is quite interesting (and also quite difficult) to find out to which extent the system uses self-modifying instructions to learn how to use self-modifying instructions. Figure 6 gives a vague idea of what is going on by showing a typical plot of the frequency of *IncProb* instructions during system life (sampled at intervals of 10^5 basic cycles). Soon after its birth, the system found it useful to dramatically increase the frequency of *IncProb*, near its death (when there was nothing more to learn) it significantly reduced this frequency. This is reminiscent of Schmidhuber's work (1974) on self-adjusting mutation rates. One major novelty is the adaptive, highly non-uniform distribution of self-modifications on "promising" individual policy components.

Stack evolution. The temporary ups and downs of the stack reflect that as the tasks change, the system selectively keeps still useful old modifications (corresponding to information conveyed by previous tasks that is still valuable for solving the current task), but deletes modifications that are too much tailored to previous tasks. In the end, there are only about 200 stack entries corresponding to only 200 *valid* probability modifications – this is a small number compared to the about $1 * 10^9$ self-modifications executed during system life.

4.3. Other Experiments with IS: Overview

In the previous experiments the learner's environment changed because of externally induced task changes. We also did experimental case studies where the tasks change in a less obvious way because of other changing learners.



133

Figure 4. Snapshot of parts of policy and storage. IP currently points to program cell 28. The integer sequence 1 2 0 (generated according to the policy's current probability distributions) will be interpreted as Subt(1, 2, 0) *— subtract the contents of work cell 2 from the contents of work cell 1 and put the result into work cell 0.*

Figure 5. The final state of the probability matrix for the Sometimes *learning problem. Gray scales indicate the magnitude of probabilities of instructions and parameters. The matrix was computed by self-modification sequences generated according to the matrix itself (initially, all probability distributions were maximum entropy distributions).*

Gödel Machines: Self-Referential Universal Problem Solvers (2003)

arXiv:cs/0309048v1 [cs.LO] 25 Sep 2003

TR IDSIA-19-03, Version 1.0; arXiv:cs.LO/0309048 v1, 25 September 2003

Gödel Machines: Self-Referential Universal Problem Solvers Making Provably Optimal Self-Improvements

Jürgen Schmidhuber
IDSIA, Galleria 2, 6928 Manno-Lugano, Switzerland
juergen@idsia.ch - http://www.idsia.ch/~juergen

Abstract

A Gödel machine solves general computational problems in a possibly stochastic and reactive environment. Its initial software includes an axiomatic description of (1) the Gödel machine's hardware, (2) known aspects of the environment, (3) goals and rewards to be achieved, (4) costs of actions and computations, (5) the initial software itself (no circularity involved here). It also includes a possibly sub-optimal initial problem-solving policy and a proof searcher searching the space of computable proof techniques—that is, programs whose outputs are proofs. Unlike previous approaches, the self-referential Gödel machine will *rewrite any* part of its software (including axioms and proof searcher) as soon as it has found a proof that this will improve its future performance. By definition, it produces optimal self-improvements, given arbitrary formalized problems and typically limited computational resources; its optimality notion is not restricted to the concept of asymptotic optimality. To initialize the proof searcher we may use the recent Optimal Ordered Problem Solver.

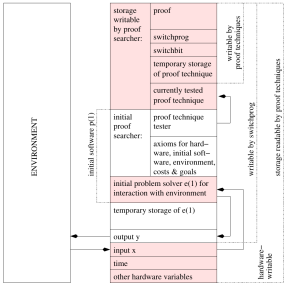
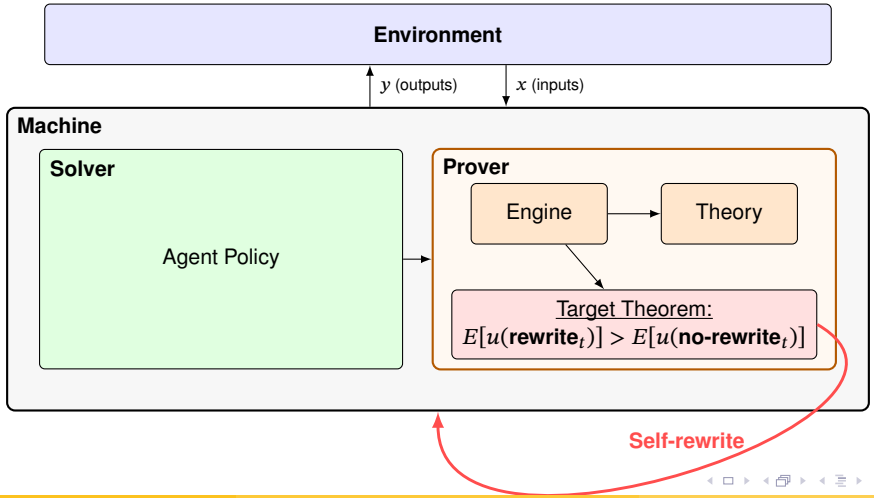


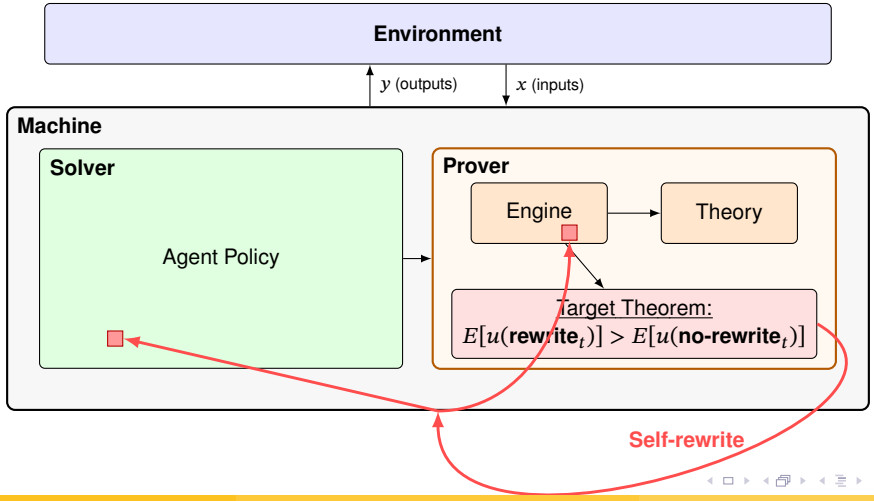
Figure 1: Storage snapshot of a not yet self-improved example Gödel machine, with the initial software still intact. See text for details.

27

Gödel Machine: architecture (simplified)



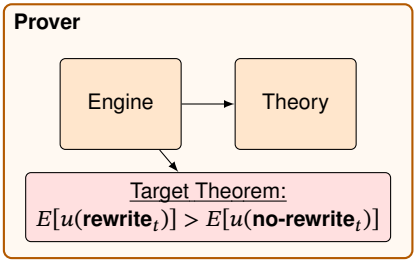
Gödel Machine: architecture (simplified)



The Global Optimality Theorem

Theorem 4.1

[...] assuming consistency of the underlying formal system [...] the utility of starting the execution of the present [rewrite] is higher than the utility of waiting for the proof searcher to produce an alternative [rewrite] later.



Difficulties:

- Theory must be consistent
- Engine must find proof of target theorem

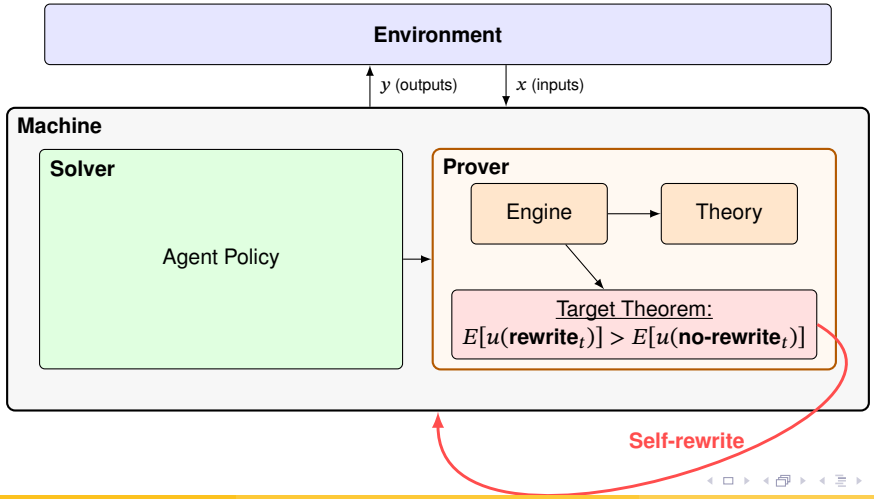
Advantages:

- Must prove comparison, not actual expectations.

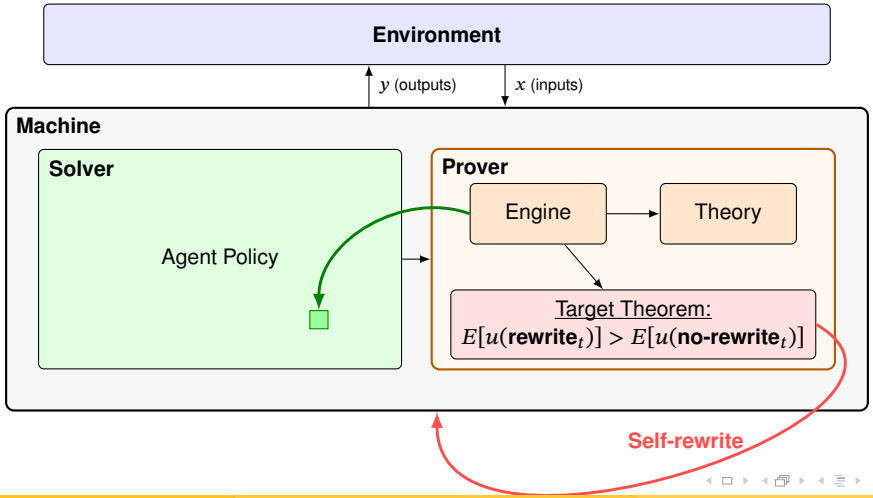
What is in the theory?

- Probability theory
- Specification of the utility
- Specification of the hardware
- Specification of the software (initial memory content, including the theory itself)
- Specification of the environment (epistemology if uncertain)
- Machine state itself! (proprioception)

What is in the theory?



What is in the theory?



What is in the engine?

- Get axioms from the theory:

```
(get_axiom 42)     ; Retrieve 42th axiom (first Kolmogorov axiom)

⇝

(: K1                          ; Symbol of first Kolmogorov axiom
   (-> (: $o Set)              ; Premise 1: $o is a sample space
       (: $f (Field $o))       ; Premise 2: $f is a field
       (: $p (Measure $f))     ; Premise 3: $p is a probability measure
       (: $e $f)               ; Premise 4: $e is an event
       (<= 0 ($p $e))))        ; Conclusion: ($p $e) is non-negative
```

- Get proprioceptive axioms:

```
(get_proprio_axiom_at_cell 4095)  ; Retrieve memory content of
                                  ; cell 4095 and deliver it as axiom

⇝

(: MC4095@2026-11-05:06:58:12:345
   (= (MemoryContent 4095 2026-11-05:06:58:12:345) 256))
```

What is in the engine?

- Chainer:

```
;; Call chainer on target theorem
(chain PARAMETERS (: $proof (>= (E (u (rewrite $r $t))) (E (u no_rewrite)))))
```

⇝

```
;; Actual proof of target theorem (with rewrite and time instantiated)
(: PROOF (> (E (u (rewrite REWRITE TIME))) (E (u no_rewrite))))
```

What is in the engine?

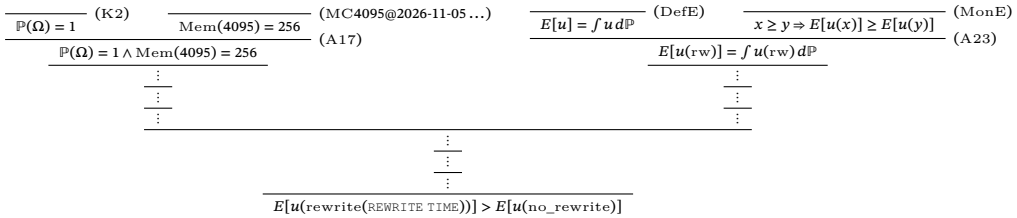
- Chainer:

```
;; Call chainer on target theorem
(chain PARAMETERS (: $proof (>= (E (u (rewrite $r $t))) (E (u no_rewrite)))))
```

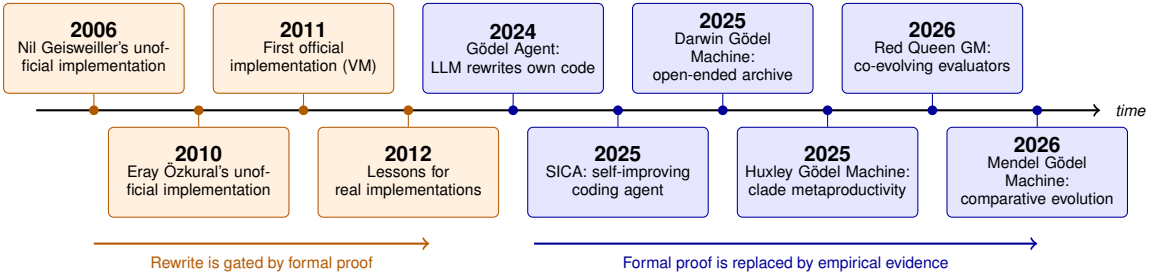
⇝

```
;; Actual proof of target theorem (with rewrite and time instantiated)
(: PROOF (> (E (u (rewrite REWRITE TIME))) (E (u no_rewrite))))
```

PROOF (illustrative):

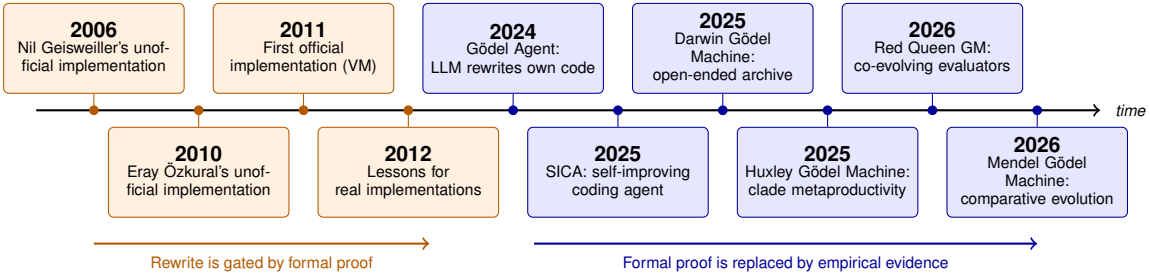


Gödel Machine Implementations



Steunebrink & Schmidhuber (2011-2012); Yin et al. (2024); Robeyns et al. (2025); Zhang et al. (2025); Wang et al. (2025); Iacob et al. (2026); Liu et al. (2026).

Gödel Machine Implementations



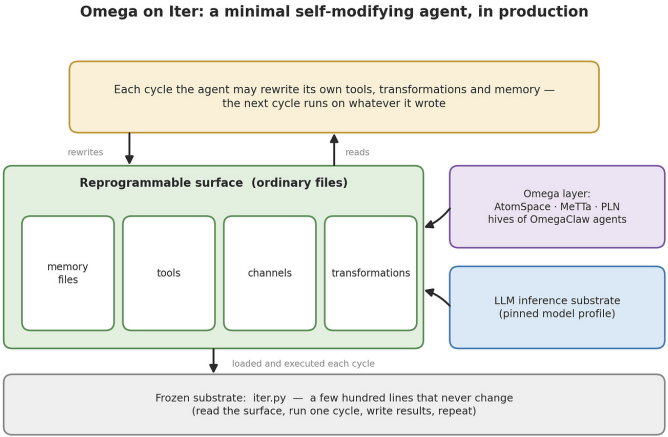
Steunebrink & Schmidhuber (2011-2012); Yin et al. (2024); Robeyns et al. (2025); Zhang et al. (2025); Wang et al. (2025); Iacob et al. (2026); Liu et al. (2026).

Relax target theorem: $\mathcal{P}(E[u(\textbf{rewrite}_t)] > E[u(\textbf{no-rewrite}_t)])$

Discover and perform rewrites using LLMs

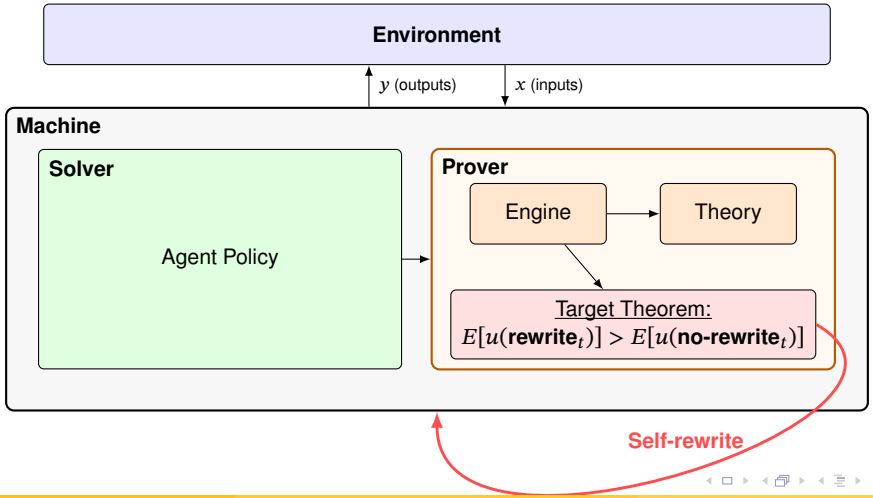
Some parts never get rewritten

Omega on Iter

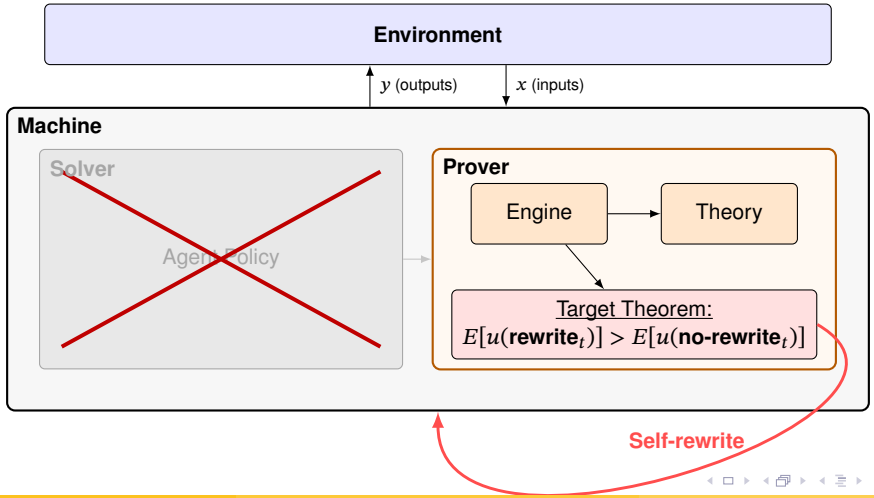


Ben Goertzel, Patrick Hammer et al.

Formal Reasoning Based Agents

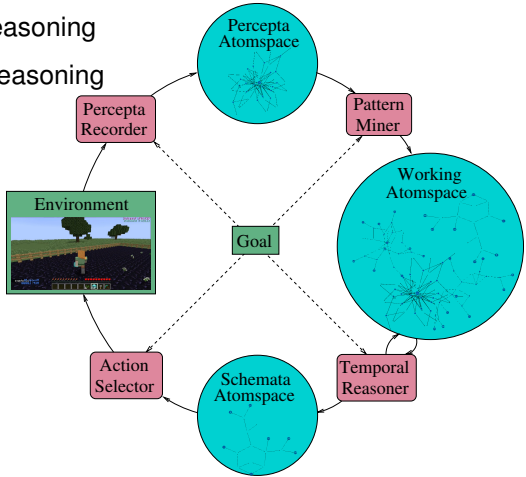


Formal Reasoning Based Agents



Formal Reasoning Based Agents

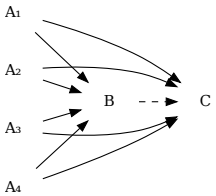
- Learning: inductive reasoning
- Planning: deductive reasoning

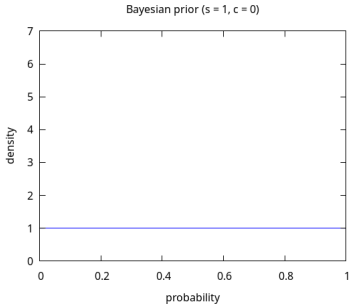


ROCCA: Rational OpenCog Controlled Agent, Nil Geisweiller & Hedra Yusuf (2023)

Formal Reasoning Based Agents: Learning via inductive reasoning

Induction + Revision in PLN:

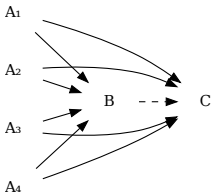


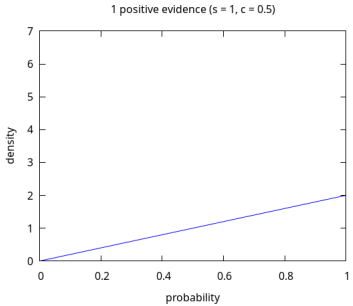


$$\dfrac{\dfrac{\dfrac{A_1 \to C \quad A_1 \to B}{B \to C \stackrel{m}{=} \langle 1\ 0.5\rangle}\ \text{(Ind)} \quad \dfrac{A_2 \to C \quad A_2 \to B}{B \to C \stackrel{m}{=} \langle 1\ 0.5\rangle}\ \text{(Ind)}}{B \to C \stackrel{m}{=} \langle 1\ 0.67\rangle}\ \text{(Rev)} \quad \dfrac{A_3 \to C \quad A_3 \to B}{B \to C \stackrel{m}{=} \langle 1\ 0.5\rangle}\ \text{(Ind)}}{B \to C \stackrel{m}{=} \langle 1\ 0.75\rangle}\ \text{(Rev)} \quad \dfrac{A_4 \to C \quad A_4 \to B}{B \to C \stackrel{m}{=} \langle 1\ 0.5\rangle}\ \text{(Ind)}}{B \to C \stackrel{m}{=} \langle 1\ 0.8\rangle}\ \text{(Rev)}$$

Formal Reasoning Based Agents: Learning via inductive reasoning

Induction + Revision in PLN:

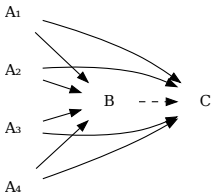


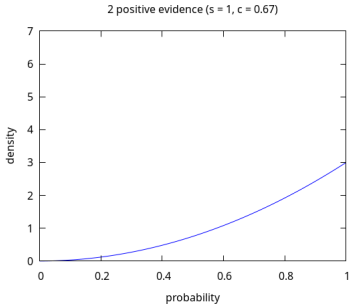


$$\dfrac{\dfrac{\dfrac{\dfrac{A_1 \to C \quad A_1 \to B}{B \to C \stackrel{m}{=} \langle 1\ 0.5\rangle}\ (\text{Ind}) \quad \dfrac{A_2 \to C \quad A_2 \to B}{B \to C \stackrel{m}{=} \langle 1\ 0.5\rangle}\ (\text{Ind})}{B \to C \stackrel{m}{=} \langle 1\ 0.67\rangle}\ (\text{Rev}) \quad \dfrac{A_3 \to C \quad A_3 \to B}{B \to C \stackrel{m}{=} \langle 1\ 0.5\rangle}\ (\text{Ind})}{B \to C \stackrel{m}{=} \langle 1\ 0.75\rangle}\ (\text{Rev}) \quad \dfrac{A_4 \to C \quad A_4 \to B}{B \to C \stackrel{m}{=} \langle 1\ 0.5\rangle}\ (\text{Ind})}{B \to C \stackrel{m}{=} \langle 1\ 0.8\rangle}\ (\text{Rev})$$

Formal Reasoning Based Agents: Learning via inductive reasoning

Induction + Revision in PLN:

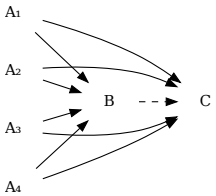


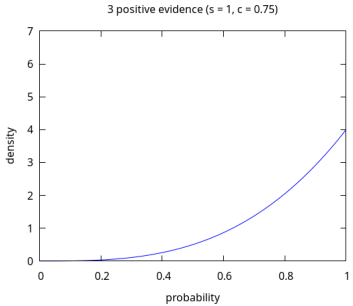


$$\dfrac{\dfrac{\dfrac{\dfrac{A_1 \to C \quad A_1 \to B}{B \to C \,\text{ⅲ}\, \langle 1\ 0.5\rangle}\,(\text{Ind}) \quad \dfrac{A_2 \to C \quad A_2 \to B}{B \to C \,\text{ⅲ}\, \langle 1\ 0.5\rangle}\,(\text{Ind})}{B \to C \,\text{ⅲ}\, \langle 1\ 0.67\rangle}\,(\text{Rev}) \quad \dfrac{A_3 \to C \quad A_3 \to B}{B \to C \,\text{ⅲ}\, \langle 1\ 0.5\rangle}\,(\text{Ind})}{B \to C \,\text{ⅲ}\, \langle 1\ 0.75\rangle}\,(\text{Rev}) \quad \dfrac{A_4 \to C \quad A_4 \to B}{B \to C \,\text{ⅲ}\, \langle 1\ 0.5\rangle}\,(\text{Ind})}{B \to C \,\text{ⅲ}\, \langle 1\ 0.8\rangle}\,(\text{Rev})$$

Formal Reasoning Based Agents: Learning via inductive reasoning

Induction + Revision in PLN:

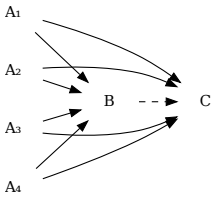


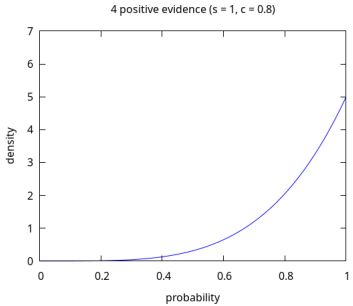


$$\dfrac{\dfrac{\dfrac{\dfrac{A_1 \to C \quad A_1 \to B}{B \to C \stackrel{m}{=} \langle 1\ 0.5\rangle}\ (\text{Ind}) \quad \dfrac{A_2 \to C \quad A_2 \to B}{B \to C \stackrel{m}{=} \langle 1\ 0.5\rangle}\ (\text{Ind})}{B \to C \stackrel{m}{=} \langle 1\ 0.67\rangle}\ (\text{Rev}) \quad \dfrac{A_3 \to C \quad A_3 \to B}{B \to C \stackrel{m}{=} \langle 1\ 0.5\rangle}\ (\text{Ind})}{B \to C \stackrel{m}{=} \langle 1\ 0.75\rangle}\ (\text{Rev}) \quad \dfrac{A_4 \to C \quad A_4 \to B}{B \to C \stackrel{m}{=} \langle 1\ 0.5\rangle}\ (\text{Ind})}{B \to C \stackrel{m}{=} \langle 1\ 0.8\rangle}\ (\text{Rev})$$

Formal Reasoning Based Agents: Learning via inductive reasoning

Induction + Revision in PLN:





$$\dfrac{\dfrac{\dfrac{\dfrac{A_1 \to C \quad A_1 \to B}{B \to C \stackrel{m}{=} \langle 1\ 0.5\rangle}\ \text{(Ind)} \quad \dfrac{A_2 \to C \quad A_2 \to B}{B \to C \stackrel{m}{=} \langle 1\ 0.5\rangle}\ \text{(Ind)}}{B \to C \stackrel{m}{=} \langle 1\ 0.67\rangle}\ \text{(Rev)} \quad \dfrac{A_3 \to C \quad A_3 \to B}{B \to C \stackrel{m}{=} \langle 1\ 0.5\rangle}\ \text{(Ind)}}{B \to C \stackrel{m}{=} \langle 1\ 0.75\rangle}\ \text{(Rev)} \quad \dfrac{A_4 \to C \quad A_4 \to B}{B \to C \stackrel{m}{=} \langle 1\ 0.5\rangle}\ \text{(Ind)}}{B \to C \stackrel{m}{=} \langle 1\ 0.8\rangle}\ \text{(Rev)}$$

Formal Reasoning Based Agents: Learning via inductive reasoning

Learn cognitive schematics via inductive reasoning:

$$\text{Precondition} \wedge \text{Action} \rightsquigarrow^{T} \text{Postcondition}$$

Formal Reasoning Based Agents: Learning via inductive reasoning

Learn cognitive schematics via inductive reasoning:

$$\text{Precondition} \wedge \text{Action} \rightsquigarrow^{T} \text{Postcondition}$$

$$\text{Outside} \wedge \text{Enter} \rightsquigarrow^{[1,5]} \text{Inside} \stackrel{\text{m}}{=} \langle 0.9\ 0.8 \rangle$$

$$(\text{Hold Key}) \wedge \text{Drop} \rightsquigarrow^{1} \neg(\text{Hold Key}) \stackrel{\text{m}}{=} \langle 0.99\ 0.75 \rangle$$

$$(\text{Hold Nothing}) \wedge (\text{Near Key}) \wedge (\text{Grab Key}) \rightsquigarrow^{2} (\text{Hold Key}) \stackrel{\text{m}}{=} \langle 0.85\ 0.9 \rangle$$

Formal Reasoning Based Agents: Planning via deductive reasoning

Plan via deductive reasoning:

$$\text{Context} \wedge \text{ActionPlan} \rightsquigarrow^{T} \text{Goal}$$

Formal Reasoning Based Agents: Planning via deductive reasoning

Plan via deductive reasoning:

$$\text{Context} \wedge \text{ActionPlan} \rightsquigarrow^{T} \text{Goal}$$

$$\dfrac{\dfrac{\textit{outside(house)} \wedge \textit{get(key)} \rightsquigarrow^{1} \textit{outside(house)} \qquad \textit{outside(house)} \wedge \textit{get(key)} \rightsquigarrow^{1} \textit{hold(key)}}{\textit{outside(house)} \wedge \textit{get(key)} \rightsquigarrow^{1} \textit{outside(house)} \wedge \textit{hold(key)}}\,(\text{CC}) \qquad \textit{outside(house)} \wedge \textit{hold(key)} \wedge \textit{go(house)} \rightsquigarrow^{1} \textit{inside(house)}}{\textit{outside(house)} \wedge \textit{get(key)} \curlywedge^{1} \textit{go(house)} \rightsquigarrow^{2} \textit{inside(house)}}\,(\text{PD})$$

Formal Reasoning Based Agents: Planning via deductive reasoning

Plan via deductive reasoning:

$$\text{Context} \wedge \text{ActionPlan} \rightsquigarrow^{T} \text{Goal}$$

$$\dfrac{\dfrac{\textit{outside(house)} \wedge \textit{get(key)} \rightsquigarrow^{1} \textit{outside(house)} \qquad \textit{outside(house)} \wedge \textit{get(key)} \rightsquigarrow^{1} \textit{hold(key)}}{\textit{outside(house)} \wedge \textit{get(key)} \rightsquigarrow^{1} \textit{outside(house)} \wedge \textit{hold(key)}}\,(\text{CC}) \qquad \textit{outside(house)} \wedge \textit{hold(key)} \wedge \textit{go(house)} \rightsquigarrow^{1} \textit{inside(house)}}{\textit{outside(house)} \wedge \textit{get(key)} \nearrow^{1} \textit{go(house)} \rightsquigarrow^{2} \textit{inside(house)}}\,(\text{PD})$$

Introspective meta-planning:

$$\text{Context} \wedge (\text{A} \nearrow \text{FindPlan}) \rightsquigarrow^{T} \text{Goal}$$

Formal Reasoning Based Agents: Planning via deductive reasoning

Plan via deductive reasoning:

$$\text{Context} \wedge \text{ActionPlan} \rightsquigarrow^{T} \text{Goal}$$

$$\dfrac{\dfrac{\textit{outside(house)} \wedge \textit{get(key)} \rightsquigarrow^{1} \textit{outside(house)} \qquad \textit{outside(house)} \wedge \textit{get(key)} \rightsquigarrow^{1} \textit{hold(key)}}{\textit{outside(house)} \wedge \textit{get(key)} \rightsquigarrow^{1} \textit{outside(house)} \wedge \textit{hold(key)}}\,(\text{CC}) \qquad \textit{outside(house)} \wedge \textit{hold(key)} \wedge \textit{go(house)} \rightsquigarrow^{1} \textit{inside(house)}}{\textit{outside(house)} \wedge \textit{get(key)} \nearrow^{1} \textit{go(house)} \rightsquigarrow^{2} \textit{inside(house)}}\,(\text{PD})$$

Introspective meta-planning:

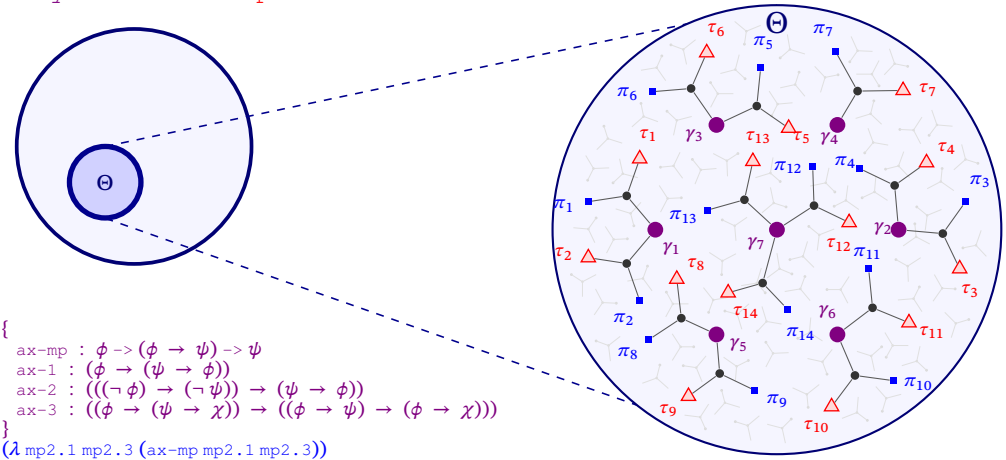
$$\text{Context} \wedge (\text{A} \nearrow \text{FindPlan}) \rightsquigarrow^{T} \text{Goal}$$

Full-on self-modification:

$$\text{Context} \wedge (\text{Rewrite Code}) \rightsquigarrow^{T} \text{Goal}$$

Reasoning Based Inference Control

$\Theta \subset \texttt{Theory} \times \texttt{Proof} \times \texttt{Proposition}$



Example:

γ = {

```
ax-mp : ϕ -> (ϕ → ψ) -> ψ
ax-1 : (ϕ → (ψ → ϕ))
ax-2 : (((¬ ϕ) → (¬ ψ)) → (ψ → ϕ))
ax-3 : ((ϕ → (ψ → χ)) → ((ϕ → ψ) → (ϕ → χ)))
```

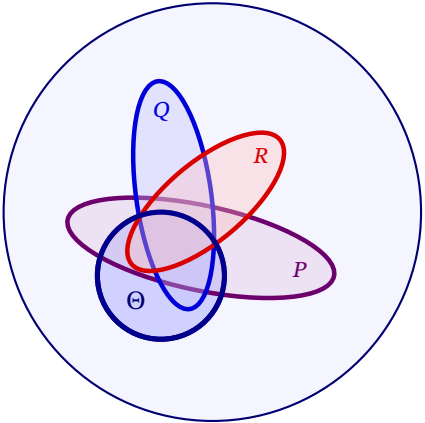
}

π = $(\lambda$ mp2.1 mp2.3 (ax-mp mp2.1 mp2.3))

τ = ϕ -> $(\phi \rightarrow (\psi \rightarrow \chi))$ -> $(\psi \rightarrow \chi)$

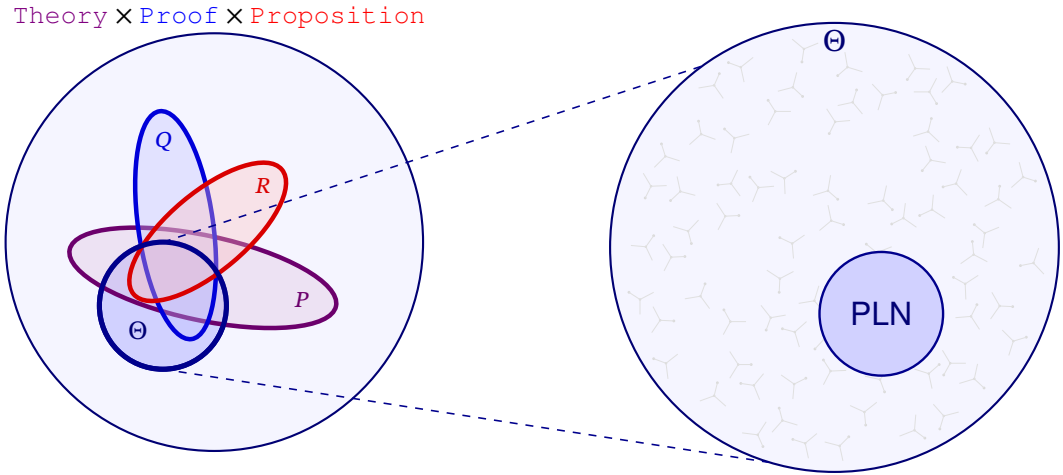
Inference Control using PLN

Theory × Proof × Proposition



$P(\gamma) \wedge Q(\pi) \wedge R(\tau) \rightarrow \Theta(\gamma, \pi, \tau) \stackrel{m}{=} <0.7\ 0.9>$

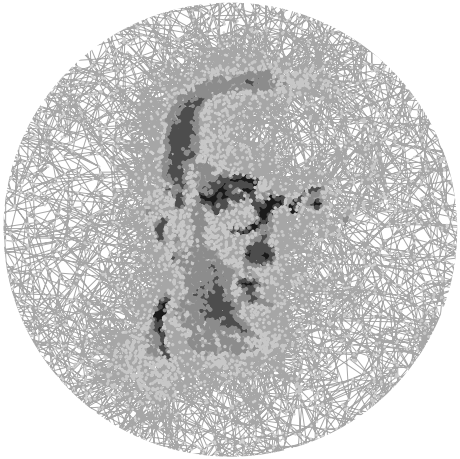
Inference Control using PLN... of PLN itself!



$$P(\gamma) \wedge Q(\pi) \wedge R(\tau) \rightarrow \Theta(\gamma, \pi, \tau) \stackrel{m}{=} <0.7\ 0.9>$$

Automated Theorem Proving

- Synthesize formal proofs and programs (Lean, Rocq, etc)
- Synthesize formal proofs and processes (f1r3lang)
- Coding formal specifications
- Autoformalization



Global Gödel Brain

Zar Goertzel

References

- J. Schmidhuber. *Evolutionary Principles in Self-Referential Learning.* Diploma thesis, TU München, 1987.
- J. Schmidhuber. *On Learning How to Learn Learning Strategies.* Technical Report FKI-198-94, TU München, 1994.
- J. Schmidhuber, J. Zhao, M. Wiering. *Shifting Inductive Bias with Success-Story Algorithm, Adaptive Levin Search, and Incremental Self-Improvement.* Machine Learning 28(1):105–130, 1997.
- J. Schmidhuber. *Gödel Machines: Self-Referential Universal Problem Solvers Making Provably Optimal Self-Improvements.* arXiv:cs/0309048 (v1 2003, v5 2006); TR IDSIA-19-03.
- B. R. Steunebrink, J. Schmidhuber. *A Family of Gödel Machine Implementations.* AGI 2011, LNCS 6830:275–280, Springer.
- B. R. Steunebrink, J. Schmidhuber. *Towards an Actual Gödel Machine Implementation: a Lesson in Self-Reflective Systems.* In *Theoretical Foundations of Artificial General Intelligence*, Atlantis Thinking Machines 4, Springer, 2012.
- X. Yin, X. Wang, L. Pan, L. Lin, X. Wan, W. Y. Wang. *Gödel Agent: A Self-Referential Agent Framework for Recursively Self-Improvement.* arXiv:2410.04444, 2024.
- M. Robeyns, M. Szummer, L. Aitchison. *A Self-Improving Coding Agent.* arXiv:2504.15228, 2025.
- J. Zhang, S. Hu, C. Lu, R. Lange, J. Clune. *Darwin Gödel Machine: Open-Ended Evolution of Self-Improving Agents.* arXiv:2505.22954, 2025.
- W. Wang, P. Piękos, L. Nanbo, F. Laakom, Y. Chen, M. Ostaszewski, M. Zhuge, J. Schmidhuber. *Huxley-Gödel Machine: Human-Level Coding Agent Development by an Approximation of the Optimal Self-Improving Machine.* arXiv:2510.21614, 2025.
- V. Iacob et al. *The Red Queen Gödel Machine: Co-Evolving Agents and Their Evaluators.* arXiv:2606.26294, 2026.
- C. Liu, Y. Liu, S. Yan, V. Tresp, Y. Ma. *Mendel Gödel Machine: Recursive Self-Improving Coding Agents via Comparative Evolution.* arXiv:2608.07645, 2026.
- B. Goertzel. *Harmonizing Progress and Safety in Recursive AI Self-Improvement.* Blog post, 2026. https://bengoertzel.substack.com/p/harmonizing-progress-and-safety-in
- N. Geisweiller, H. S. Yusuf. *ROCCA: Rational OpenCog Controlled Agent.* AGI, 2023.
- N. Geisweiller, H. S. Yusuf. *Probabilistic Logic Networks for Temporal and Procedural Reasoning.* AGI, 2023.
- N. Geisweiller. *Estimating the Probability of a Conjecture to be a Theorem with PLN for Inference Control.* AITP, 2025.